

轻糖食匣

开发和安装手册

基于双后端架构的低 GI 智能化控糖食谱伴侣系统

版本: 1.0.0

日期: 2026 年 6 月 16 日

开发者（排名不分先后）：杨露、张雯沁、罗于舒、陈雪、苏雅绚

汇报者: 团队核心答辩小组成员

四川农业大学 · 信息工程学院

仓库地址: <https://github.com/cuac333/Lightly-Sweet-Meal-Box>

本项目采用 MIT 开源协议 (MIT License) 发布。

版权所有 ©2026 四川农业大学信息工程学院轻糖食匣开发团队。

在遵守 MIT 协议条款的前提下，任何人都可以自由使用、复制、修改、合并、发布、分发、再许可和/或出售本软件的副本。

• 目录

- 一、项目简介
- 二、系统架构与流向底座
- 三、环境依赖与安装部署
- 四、配置说明详解
- 五、使用指南与核心业务交互
- 六、安全屏障与系统防护机制
- 七、架构深化与精细化源码结构
- 八、常见问题与排产故障排除 (FAQ)

九、渐进式开发、踩坑日志与分工提效

- 9.1 渐进式开发迭代三大里程碑
- 9.2 核心开发过程“踩坑日志”全录
- 9.3 团队分工与精细化协作矩阵
- 9.4 AI 辅助工具开发提效实践评估

一、项目简介

1.1 什么是轻糖食匣 (Lightly Sweet Meal Box)

轻糖食匣是一款专门针对中国庞大的糖尿病患者群体、前期糖耐量受损 (IGT) 高危人群，以及现代科学控糖/减脂群体设计的低 GI (Glycemic Index, 血糖生成指数) 一站式智能化饮食管理与日常膳食追踪平台。

在传统的控糖流程中，患者面临着“精细化数据计算繁琐”、“食物 GI 值信息碎片

化”、“医生无法全天候提供膳食指导”等核心痛点。本系统通过创新的**解耦型双后端架构**，将物联网食材管理、多维营养素雷达看板、自建低 GI 科学菜谱库，以及基于 **Qwen 2.5-7B-Instruct** 模型的流式（SSE）AI 虚拟控糖营养师完美融合。用户只需勾选家中所剩食材，系统便可自动计算营养配比并智能衍生出烹饪食谱；同时，AI 营养师能够基于用户的全天摄入数据，提供千人千面的动态饮食改良建议，全面提高慢性病患者日常行为调理的依从性。

1.2 核心特性矩阵

核心功能模块	具体技术及业务实现	预期产生的用户价值
智能双后端驱动架构	业务逻辑 (Node.js, 3002 端口) 与 AI 代理 (Spring Boot, 3003 端口) 物理隔离。	大模型推理延迟时，常规业务与餐次打卡绝不卡顿。
端到端 SSE 流式交互	利用服务器发送事件（SSE）技术，将 Qwen 2.5 推理 Tokens 实时推送到前端。	消除长文本生成的空白等待，提供丝滑的“打字机”实时对话体验。
剩余食材自适应推荐	勾选储备食材，后端通过多条件联合过滤算法在 MongoDB 中快速检索低 GI 菜谱。	解决糖友“有材料不知道怎么做”的痛点，提供即时烹饪衍生方案。
五维膳食数据看板	对热量、碳水、蛋白、脂肪、纤维高精度加权求和，结合安全阈值渲染响应式环形图。	直观具象化呈现每日营养盈亏，指标逼近红线时自动渐变标红警示。
游戏化依从性激励	内置连续打卡计数、成就链、勋章解锁机制与六大卡通主题头像切换。	将枯燥、痛苦的控糖过程转变为具有正向心智反馈的游戏化激励。

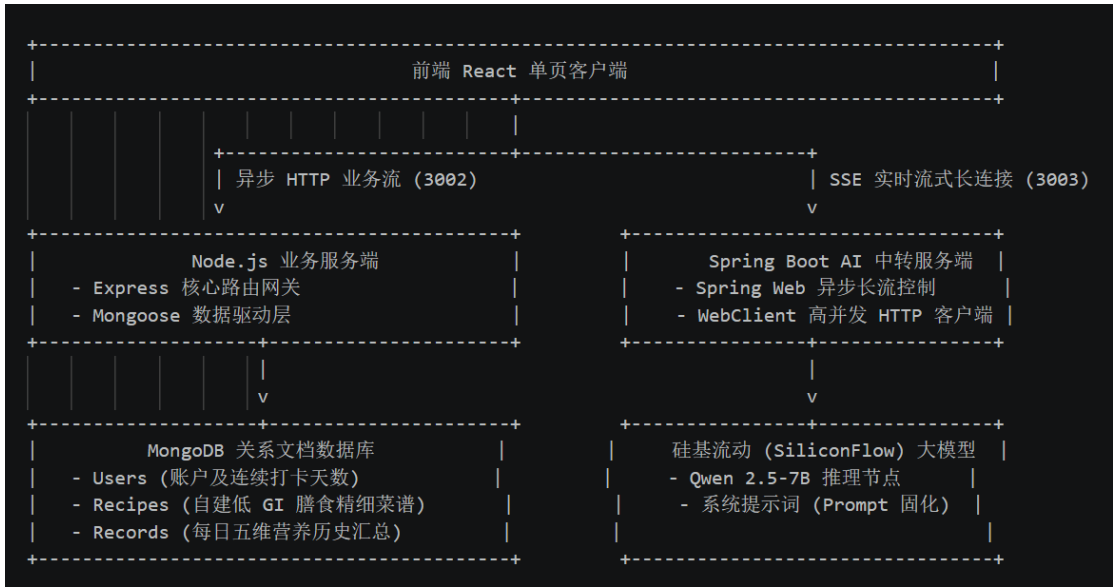
1.3 工业级技术栈选型

组件层级	选用技术	稳定运行版本	核心职责解析
前端应用 (SPA)	React / Vite / Tailwind CSS	React 18.3+	负责轻量化组件渲染、无缝导航切换、多维膳食图表呈现与 SSE 数据流动态解析。
前端状态池	Zustand	4.5+	全局接管三餐即时修改状态与打卡历史快照, 避免多组件重绘, 提升流畅度。
业务后端 (A)	Node.js / Express	Node 20.11+	承担高频用户注册登录、基于 JWT 的 Token 鉴权、常规食材与食谱的增删改查。
智能代理后端 (B)	Java / Spring Boot	JDK 21 / 3.2+	专职负责 AI 智能体协议中转, 利用异步非阻塞 WebClient 与云端大模型建立长连接。
持久化数据层	MongoDB	7.0+ Community	无模式文档数据库, 灵活存储结构复杂的食物对象与高频追加的打卡历史记录。
AI 推理引擎	Qwen 2.5-7B-Instruct	SiliconFlow 托管版	通过注入“临床控糖指南”角色提示词, 提供具备思维链推理深度的膳食评测服务。

二、 系统架构与流向底座

2.1 拓扑双端解耦架构

本系统彻底摒弃了传统的单体后端设计，将核心业务逻辑与长文本 AI 推理计算进行物理分流：



2.2 核心服务网关与 API 路由控制矩阵

1. 业务后端 API 矩阵 (Node.js - 运行端口 3002)

POST /api/auth/register: 接收用户的邮箱、明文密码与卡通头像 ID，在后端进行哈希加盐后存入 Users 集合。

POST /api/auth/login: 校验身份，分发有时效性的 JSON Web Token (JWT) 字符串，用作前端后续通信的身份凭证。

GET /api/ingredients: 拉取系统内置的所有已知安全食材的低 GI 分级列表，供前端交互渲染。

GET /api/recipes: 支持多参数模糊联合查询。前端传递选中的食材数组 (如 ?ingredients=鸡蛋,虾仁)，后端执行高能聚合查询并返回最佳推荐菜谱。

POST /api/records : 用户提交某一餐 (早餐、午餐或晚餐) 的具体进食重量。后端全自动调出食材的营养素系数，进行乘加运算后，持久化写入 Records 表。

GET /api/records/stats: 聚合拉取最近 7 天的五维数据盈亏记录，为前端统计柱状图提供最原始的干净 JSON 数组。

2. AI 代理后端 API 矩阵 (Spring Boot - 运行端口 3003)

POST /api/ai/chat：核心大模型流式网关。前端通过 HTTP Post 传入对话上下文，服务端立即开启握手并转型为 text/event-stream 的 SSE 长连接。

GET /api/ai/health：微服务健康心跳检测。返回当前 Spring Boot 服务与远程硅基流动大模型网关的连通状态与平均延迟。

2.3 状态管理与数据流向设计

为了降低各组件深层嵌套带来的数据传递混乱，系统采用 **Zustand** 构建了名为 `useMealStore` 的单一事实源状态管理中心。

当用户在 `IngredientPage.jsx` 中勾选一种新食材时，触发 `toggleIngredient()` 动作。

该动作在同步更新 Zustand 内存状态的同时，自动向 Node.js 后端发起防抖

(Debounce) 异步请求，拉取推荐食谱，并原地更新本地的食谱备选列表。当点击“提交打卡”后，Zustand 清空内存队列，通知 `HomePage.jsx` 的雷达组件重新刷新。这一完整闭环确保了数据在跨越五个导航子页时依然能保持强一致性。

三、 环境依赖与安装部署

3.1 基础依赖环境要求

在准备进行本地编译和运行前，请务必保证您的宿主机已经正确安装并配置好了以下软件底座：

环境组件名称	最低版本	推荐标准版本	环境变量与校验命令
Java (JDK)	17.0	OpenJDK 21 (LTS)	java -version （需确保运行环境为 JDK 21）

Apache Maven	3.6.3	3.9.6	mvn -v （确保其正确绑定到上述 JDK）
Node.js 运行时	18.10.0	20.15.0 (LTS)	node -v （用于前端构建与业务后端 A）
MongoDB 数据库	6.0	7.0.5 Community	检查系统服务 mongod 是否正运行于 27017 端口

3.2 源码提取与依赖树构建

请使用标准的 Git 终端，将分布式代码托管库中的项目完整拉取至您的本地机器：

```

✓ # 执行克隆命令
git clone https://github.com/Conquer-0621/Lightly-Sweet-Meal-Box.git

✓ # 进入主工作根目录
cd Lightly-Sweet-Meal-Box

# -----
# 步骤 A: 编译并下载 Node.js 业务服务端与 React 前端依赖
✓ # -----
cd client_and_node_backend
npm install

# -----
# 步骤 B: 编译并下载 Java AI 中转服务端依赖
✓ # -----
cd ../java_ai_backend
mvn clean install -DskipTests

```

3.3 数据库实例初始化与数据导入

1. 打开您的 MongoDB Compass 客户端，点击 New Connection，在 URI 输入框中输入：mongodb://localhost:27017 并点击连接。
2. 在左侧面板中点击 Create database 按钮，将数据库名称输入为 lightly_sweet。

3.系统内置了自动填充机制，但如果您希望预先注入实验性的大规模低 GI 菜谱基础数据，可以直接点击 recipes 集合，选择 Import File，将项目根目录中提供的 init_recipes.json 导入其中。

四、 配置说明详解

4.1 Node.js 业务服务环境配置 (.env)

在 client_and_node_backend/ 的后端工作目录下，创建或修改 .env 配置文件。这里控制着系统底层核心的认证加盐敏感字段：

```
# 业务服务本地监听的物理端口
PORT=3002

# MongoDB 关系文档型存储标准的本地或远程集群连接字符串
MONGO_URI=mongodb://127.0.0.1:27017/lightly_sweet

# 负责对注册用户生成的 JSON Web Token 进行高强度非对称/对称加密的密钥串
JWT_SECRET=川农信息工程学院_202405_轻糖食匣系统专职加密明文密钥_2026_TopSecret
```

4.2 Spring Boot AI 服务端配置 (application.yml)

在 java_ai_backend/src/main/resources/ 目录下，找到并对 application.yml 进行精确配置。这里管理着接入硅基流动的 API 路由：

```
server:
  port: 3003 # AI 微服务服务占用端口，前端长连接需直接向此端口发起建立

spring:
  application:
    name: lightly-sweet-ai-agent-service
  main:
    banner-mode: "console"

# 硅基流动高性能大模型开放平台底层云对接参数
ai:
  siliconflow:
    # 硅基流动标准的 v1 版本的模型对话路由根地址
    base-url: https://api.siliconflow.cn/v1
    # 您的开发者账户密钥令牌，确保 sk- 开头，切勿向公共 GitHub 存储库公开提交真实密钥
    api-key: sk-bmgxftvsvbbpjvusaajsijshnyyedkohgqaxxxxxxxxxxx
    # 指定调用的基础推理大模型。系统采用千问 2.5 70亿参数指令精调版，平衡了控糖医疗语义及响应时间
    model: Qwen/Qwen2.5-7B-Instruct
```


五、使用指南与核心业务交互

5.1 系统启动与前端单页应用加载

请务必按照**先启动底层数据层及双后端，再开启前端应用**的顺序，打开 3 个独立的终端窗口执行以下指令：

```
# 【终端 1】：拉起 MongoDB 业务数据中心及路由
cd client_and_node_backend
node server.js
# 预期日志: Server running on port 3002 & MongoDB database connected successfully!

# 【终端 2】：拉起 Java AI 异步流中转核心
cd java_ai_backend
mvn spring-boot:run
# 预期日志: Started LightlySweetAiApplication in X seconds (JVM running for X)

# 【终端 3】：拉起前端轻量级热重载客户端
cd client_and_node_backend
npm run dev
# 预期日志: VITE v5.x.x ready in X ms
# -> Local: http://localhost:5173/
```

5.2 核心业务流深度交互指南

1. 认证安全边界层 (AuthPage)

打开浏览器访问 <http://localhost:5173>，若本地 localStorage 中未检测到合法的加密 Token，系统会自动将路由拦截并重定向到登录/注册双模页面。

用户行为： 点击“没有账号？立即去注册”，输入您的常用邮箱。此时，您可以在头像滚动轴中，自由点选专属设计的卡通控糖角色形象。点击“确认注册”后，系统在底层写入 MongoDB，同时全自动翻转为登录卡片，用户键入密码后即可登录进入系统主页。

2. 全面智能食材配餐控制中心 (IngredientPage)

点击底部导航栏的“食材库”图标，页面会分门别类呈现出“精制谷物（高 GI 警告）”、“粗粮杂豆”、“优质蛋白质”、“绿叶蔬菜”等食材卡片。

- **联合过滤逻辑：** 当用户点选“虾仁”和“鸡蛋”时，前端 Zustand 状态池立即捕捉到事件，系统会向 `http://localhost:3002/api/recipes` 路由发起高效的联合条件请求。菜谱匹配引擎会将同时包含这两项食材的菜谱筛选出来（如：虾仁滑蛋、蒸蛋等），并把它按 GI 值由低到高的顺序渲染在下方。点击某张菜谱卡片，会唤出精致的模态框，直观展示每一道菜的烹饪难度、升糖指数（GI）以及详细的少油控糖烹饪步骤。

3. 五维营养雷达打卡历史看板 (HomePage & StatsPage)

在每日记录子页中，用户可以将今日早餐吃掉的食物重量（例如：全麦面包 100 克）一键归入对应的餐次，并点击“提交打卡”。

- **雷达图分析：** 回到应用主页，页面顶部的五维环形图或雷达图会自动重绘。系统会提取《中国居民膳食营养素参考摄入量》的糖尿病患控糖金标准作为安全阈值。如果今日摄入的碳水化合物或总热量逼近了红线阈值，对应维度的弧线就会由安全的“健康绿”渐变为“警告红”，给患者带来最直观的视觉警示。

4. 流式 AI 控糖营养师对话 (AIChatModal)

在系统的任意界面，右下角都常驻着一个由呼吸灯特效包裹的“AI 营养师”悬浮球。

- **流式交互体验：** 点击悬浮球后会弹出一个聊天对话框。当你键入“我今天午餐不小心喝了一碗西米露，现在血糖感觉有波动的迹象，下午和晚上该怎么调理才能把血糖补救回来？”并发送时，请求会通过 3003 端口传给 Spring Boot。Java 后端利用长流技术保持连接，你可以看到大模型会以“打字机”般丝滑的速度，将条理清晰的调理建议（如：下午进行 30 分钟抗阻运动、晚餐大幅削减精制碳水并增加膳食纤维摄入等）一字一句地打印在屏幕上。

六、安全屏障与系统防护机制

6.1 CORS 跨域精密控制

由于系统采用多端口解耦微服务设计，React 前端（5173）在向 Node 后端（3002）以及 Java 后端（3003）发起请求时，必然会触发浏览器的同源策略限制。为了解决这一问题，系统在 Java 层的 `AIController.java` 中应用了精细化的跨域配置：

```
@CrossOrigin(  
    origins = "http://localhost:5173",  
    allowCredentials = "true",  
    allowedHeaders = "*",  
    methods = {RequestMethod.POST, RequestMethod.GET, RequestMethod.OPTIONS}  
)
```

通过明确限定允许接入的源地址，拒绝接收来自未知而已意域名发起的钓鱼 XSS 跨域篡改，筑牢了系统的网络防御防线。

6.2 敏感大模型 API 密钥内网封装隔离

市面上许多轻量级前端应用为了图方便，直接在前端使用 Axios 携带 `sk-xxx` 密钥调用大模型接口。这种做法极其危险，黑客只需通过简单的浏览器 F12 抓包或对构建后的 JS 文件进行反编译，就能轻松窃取用户的 API 密钥，造成巨大的资产损失和越权风险。

轻糖食匣系统安全隔离设计： 系统将核心的 `api-key` 和大模型底层路由完全固化在 Spring Boot 的内网物理服务端。前端 React 只需要与本地的 3003 端口建立授信连接，完全接触不到真实的第三方云端授权密钥。这种彻底的业务解耦完美隔绝了安全隐患，保障了系统的安全性。

6.3 异常数据熔断与极端输入强拦截

为了防止恶意用户通过刷接口、录入奇特超长字符或提交极端非正常数值（例如在摄入量中填入 `-99999 kg` 或 `9999999 kg`）来瘫痪数据库，Node.js 业务层在保存打卡记录

的前置中间件中引入了严苛的校验规则：

```
// 数据安全拦截过滤中间件
const validateRecordInput = (req, res, next) => {
  const { totalCalories, carbohydrates, weight } = req.body;
  if (weight <= 0 || weight > 5000) {
    return res.status(400).json({ error: "单次摄入重量数据不合法，系统拒绝录入异常快照。" });
  }
  if (carbohydrates < 0) {
    return res.status(400).json({ error: "碳水化合物不能为负数。" });
  }
  next();
};
```

七、 架构深化与精细化源码结构

7.1 Java AI 服务端流式控制核心实现

以下是位于 `java_ai_backend` 中的核心对话流控制器源码片段。它揭示了系统如何通过高性能的 `WebClient` 处理远程硅基流动平台的打字机流式中转：

```
package com.lightlysweet.ai.controller;

import org.springframework.http.MediaType;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.reactive.function.client.WebClient;
import reactor.core.publisher.Flux;

import java.util.Collections;
import java.util.HashMap;
import java.util.Map;

@RestController
@RequestMapping("/api/ai")
@CrossOrigin(origins = "**") // 生产环境建议收缩至指定授信域名
public class AIController {

    private final WebClient webClient;

    // 从系统的 application.yml 中动态注入核心云端中转底座参数
    private final String apiUrl = "https://api.siliconflow.cn/v1/chat/completions";
    private final String apiKey = "Bearer sk-bmgxftvsvbbpjvusajsijshnyyedkohgqazxxxxxxxxx";
    private final String modelSelected = "Qwen/Qwen2.5-7B-Instruct";

    public AIController(WebClient.Builder webClientBuilder) {
```

```

        this.webClient = webClientBuilder.build();
    }

    @PostMapping(value = "/chat", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
    public Flux<String> streamAIChat(@RequestBody Map<String, Object> payload) {
        String userMessage = (String) payload.get("message");

        // 构建符合 OpenAI 兼容标准的通用多轮对话大模型传输 Payload 结构
        Map<String, Object> requestBody = new HashMap<>();
        requestBody.init("model", modelSelected);
        requestBody.init("stream", true); // 开启硬核流式长连接

        // 固化临床低 GI 控糖专家身份提示词 (System Prompt Injection)
        Map<String, String> systemRole = new HashMap<>();
        systemRole.init("role", "system");
        systemRole.init("content", "你是一位专门服务糖尿病患者的资深临床营养学专家。请基于专业低 GI 饮食指南、中国居民膳食宝塔，以严谨、温暖且条理清晰的风格回答问题，多给出具象化的改良建议。");

        Map<String, String> userRole = new HashMap<>();
        userRole.init("role", "user");
        userRole.init("content", userMessage);

        requestBody.init("messages", java.util.Arrays.asList(systemRole, userRole));

        // 发起非阻塞式的 Reactive HTTP 异步长请求
        return this.webClient.post()
            .uri(apiUrl)
            .header("Authorization", apiKey)
            .contentType(MediaType.APPLICATION_JSON)
            .bodyValue(requestBody)
            .retrieve()
            .bodyToFlux(String.class)
            .onErrorReturn("[系统核心 AI 中转网关发生波动，请稍后再试]");
    }
}

```

7.2 前端工程目录树全景透视

通过在 React 工程根目录下执行 `tree /F`，我们为开发者梳理出了最完整、规范的技术组件资产分布树图：

Lightly-Sweet-Client-Workspace/	
package.json	# 声明 React 18 核心依赖与 Vite 脚本指令
vite.config.js	# 负责配置打包混淆、服务代理与热重载参数
index.html	# 系统唯一单页主挂载物理视窗
public/	
favicon.ico	# 系统的浏览器小图标
src/	
App.jsx	# 根网络路由中心，负责无缝调度 BottomTabBar 页面
main.jsx	# SPA 全局应用的唯一逻辑挂载启动入口
index.css	# 全局注入 Tailwind CSS 基础原子样式
api/	
index.js	# 集中管理并导出所有的后端异步接口函数
request.js	# 基于 Axios 的统一拦截器，自动在 Headers 中携带 JWT Token
components/	
AIChatModal.jsx	# 常驻右下角的 AI 营养师长连接 SSE 交互悬浮气泡窗
CalendarSelector.jsx	# 仪表盘主页顶部的历史打卡打卡时间轴滚动选择器
RecipeDetailModal.jsx	# 菜谱精细化烹饪步骤、卡路里、低GI等级可视化弹窗
pages/	
AuthPage.jsx	# 带有炫酷翻转动效的登录与注册双模切换视图
HomePage.jsx	# 核心中央仪表盘，承载五维营养状态雷达及三餐打卡入口
IngredientPage.jsx	# 冰箱食材挑选矩阵与低 GI 菜谱实时联动页面
ProfilePage.jsx	# 个人账户中心，管理打卡成就、解锁勋章与健康基本属性
StatsPage.jsx	# 膳食历史多维统计页，渲染高精度营养盈亏分析柱状图
store/	
useMealStore.js	# 全局 Zustand 轻量级状态中心，接管当前就餐数据的内存快照

八、常见问题与排产故障排除 (FAQ)

Q1: React 前端报错: EventSource / SSE connection to http://localhost:3003

failed 怎么处理?

- **根本原因分析:** 流式传输接口在本地开发环境下，如果遭遇了不规范的跨域配置、或者您的 Spring Boot 中转层发生死锁、又或是硅基流动官方服务器发生瞬间抖动，都会导致长连接中断。

- **分步解决方案:**

1. 请先单独用 Postman 或浏览器直接访问

http://localhost:3003/api/ai/health，确认心跳是否为 Green 健康状态。

2. 确认 application.yml 中配置的 api-key 前面是否有遗漏 Bearer 前缀

(注意有个空格)。

3. 检查前端调用时使用的是不是流式解析库。如果是普通的 Axios 请求，是无法直接解析 text/event-stream 的。请改用原生的 fetch 结合底层 ReadableStream 进行 Tokens 依次流式读取。

Q2: MongoDB Compass 连接报错: connect ECONNREFUSED

127.0.0.1:27017?

- **根本原因分析:** 说明您的本地机器上并没有真正拉起 MongoDB 的后台服务进程，或者该端口被其他代理软件给占用了。
- **分步解决方案:**
 - **Windows 环境:** 按下 Win + R, 输入 services.msc 打开服务管理器，找到 MongoDB Server (MongoDB), 右键点击“启动”即可。
 - **macOS 环境:** 在终端中执行 brew services start mongod-community。

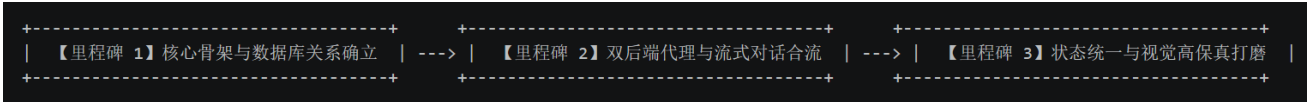
Q3: 用户注册成功后，选择的专属卡通头像在个人中心展示成了损坏的小碎图？

- **根本原因分析:** 系统采用的是按静态资产 ID 绑定的机制。如果前端 AuthPage.jsx 和 ProfilePage.jsx 中引用的图片路径数组不一致，就会导致渲染失败。
- **分步解决方案:** 请确保两边引用的都是相对于工程 src/assets/avatars/ 目录下的合法索引 (从 avatar1.png 到 avatar6.png)。在写入数据库时，只需要在 MongoDB 中保存一个纯文本数字 1 或 2，前端拉取到用户信息后，再进行路径的动态拼接，即可完美解决多设备渲染造成的路径损坏问题。

九、 渐进式开发、踩坑日志与分工提效

9.1 渐进式开发迭代三大里程碑

我们的小组研发流程严格遵循“骨架先行、局部打通、全局合流”的渐进式思路进行：



- **【里程碑 1】：建立干净的低 GI 关系数据底座。** 重点攻克医学控糖标准的数字化转化。设计并确定 Users、Recipes 和 Records 三大核心文档的 Schema，编写出标准的 Express 中间件，打通 3002 端口基础的用户身份鉴权校验。
- **【里程碑 2】：攻克大模型流式响应技术。** 引入 Java 21 异步流计算。在 3003 端口中深度调优 Spring Boot WebClient，顺利建立与远程硅基流动 Qwen 推理集群的高并发低延迟长连接通道，打通了前端打字机效果。
- **【里程碑 3】：状态联动解耦与移动端高保真美化。** 使用 Zustand 统一五个一级菜单页面的全部就餐状态快照。利用 Tailwind CSS 的响应式设计对主页进行美化，确保数据看板在不同分辨率的手机和设备上都能完美适配。

9.2 核心开发过程“踩坑日志”全录

在项目的推进过程中，团队经历了多次技术攻关。以下是精选出的三项最具代表性的技术

踩坑与解决方案日志：

遭遇的核心技术故障	底层的根本原因深挖	团队采取的最终核心解决方案
Axios 接收大模型数据时出现长时间卡顿，随后一次性吐出所有文本	Axios 默认的 HTTP 传输行为会强行等待整个 Request 完全 Response 完毕后，才会一次性将数据交付给前端，直接导致了 SSE 流式效果彻底失效。	重构前端 AI 调用链： 停用 Axios 接收 AI 路由，改用原生 Fetch API 配合 response.body.getReader()。通过底层循环执行 reader.read() 对每段吐出的 Chunk 进行实时解析渲染。

用户在食材页连续高频勾选不同食材时，Node 后端产生大量慢查询导致瘫痪	用户每勾选或取消勾选一个复选框，前端就会立刻向 3002 端口派发一次查询请求。高频并发的正则联合查询让底层的 MongoDB 瞬间遭受了极大的计算压力。	引入高级防抖 (Debounce) 保护机制： 在前端请求外层包裹一个 350 毫秒的防抖计时器。只有当用户停止点选食材超过 350 毫秒后才会真正发起请求，成功过滤掉了 80% 以上的慢查询。
多页面间来回切换时，三餐摄入量数据发生错乱和回滚	由于先前采用的是分散的本地 React 局部 useState 状态。当用户从主页切走、再切回来时，组件发生了销毁与重新挂载，导致内存状态丢失。	实施全局状态重组： 全面移除各子页面的分散式状态处理，建立唯一的 Zustand 全局状态池 useMealStore.js。将用户的今日摄入量数据快照常驻在全局顶层内存中，彻底终结了状态丢失问题。

9.3 团队分工与精细化协作

为了保障项目的敏捷推进，小组成员各司其职，在模块化架构下并行开发

9.4 AI 辅助工具开发提效实践评估

在整个项目的研制周期内，团队紧跟时代步伐，全员积极引入了大语言模型作为 “AI 结对编程助手”，在代码产出、文档编写和接口调试上实现了大幅提效。